



Contenido

01

¿Qué es un método en VR?

02

Sintaxis básica de un método

03

Voz vs Retorno

04

Proyectos y Ajustes

05

El valor de Retorno (Retorno)

06

Invocación de Métodos

07

Static vs Instanciable

08

Selección de Métodos

09

Caso práctico: Organización

10

Errores Comunes en Métodos

11

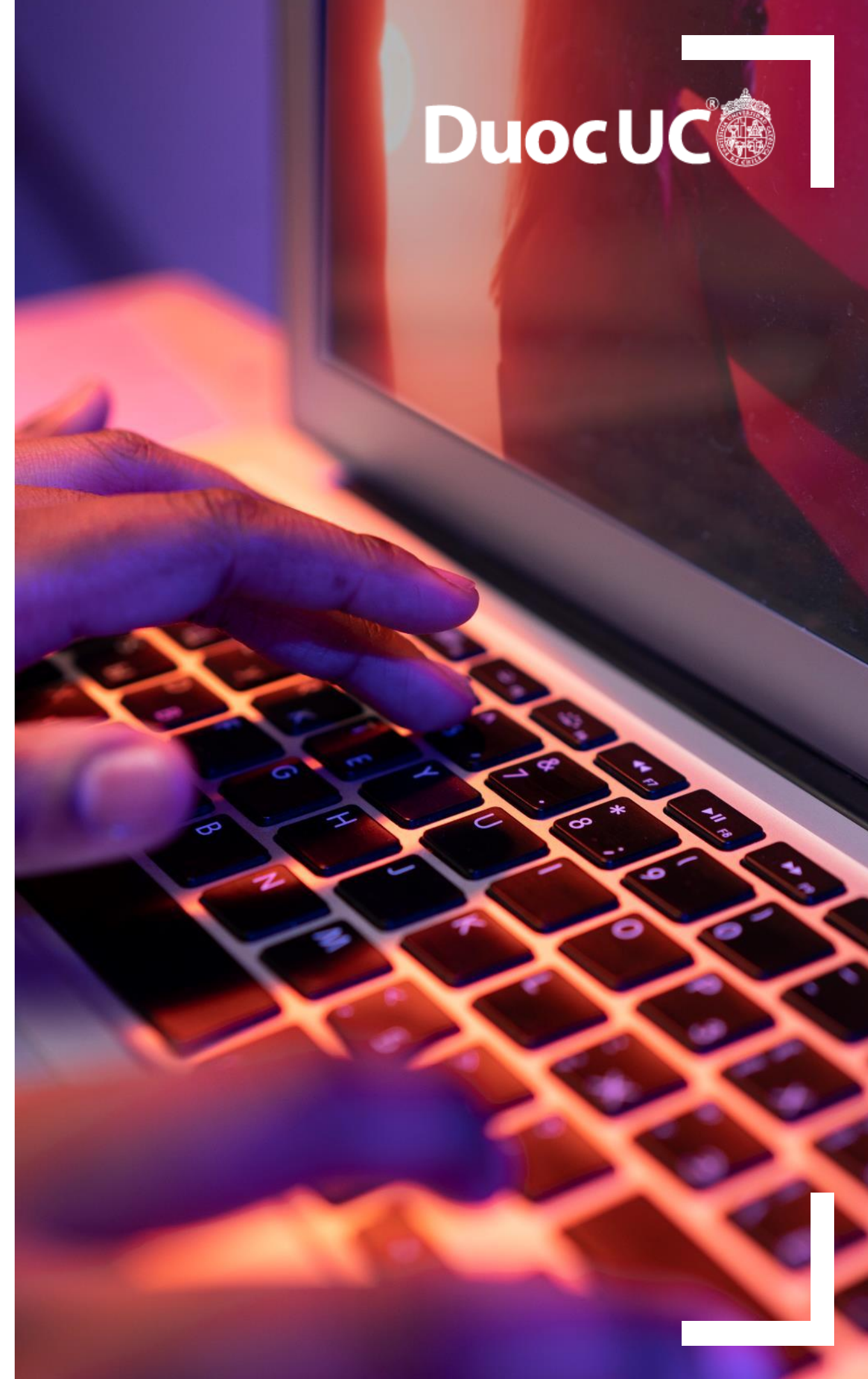
Buenas prácticas

12

Resumen Visual Métodos en VR

13

Conclusiones y próximos pasos



```
public int sumar(int a, int b) { return a + b; } public void saludar(String nombre) { System.out.println("Hola, " + nombre); } int resultado = sumar(5, 10); saludar("Mundo");
```

</> Fundamentos del Lenguaje

Java

Métodos en Java

Definir, invocar, reutilizar y organizar código.



Reutilización
saludar(), sumar()



Organización
imprimirResultado()

 MetodosBasicos.java

```
// 1. Método sin retorno (void)
public void saludar(String nombre) {
    System.out.println("Hola, " + nombre);
}

// 2. Método con retorno y parámetros
public int sumar(int a, int b) {
    return a + b;
}

// 3. Invocación de métodos
saludar("Ana");
int total = sumar(10, 20);
System.out.println("Total: " + total);
```





¿Qué es un Método en Java?

Bloques de código con un propósito específico



Definición

Un **método** es un conjunto de instrucciones agrupadas bajo un nombre, diseñadas para realizar una tarea específica cuando se le invoca.



Agrupar instrucciones

Unifica múltiples líneas de código en una sola acción lógica.



Permite reutilizar lógica

Escribes el código una vez y lo usas (llamas) todas las veces que necesites.



Mejora la organización

Divide programas complejos en piezas pequeñas y manejables.

Concepto Clave:

- En Java, toda acción o comportamiento ocurre dentro de un método.

Flujo de un Método



void vs Retorno

Tipos de retorno en los métodos Java

Tipos de Métodos

Un método puede realizar una acción sin devolver nada, o calcular un valor y devolverlo a quien lo llamó usando la palabra clave `return`.



Métodos void

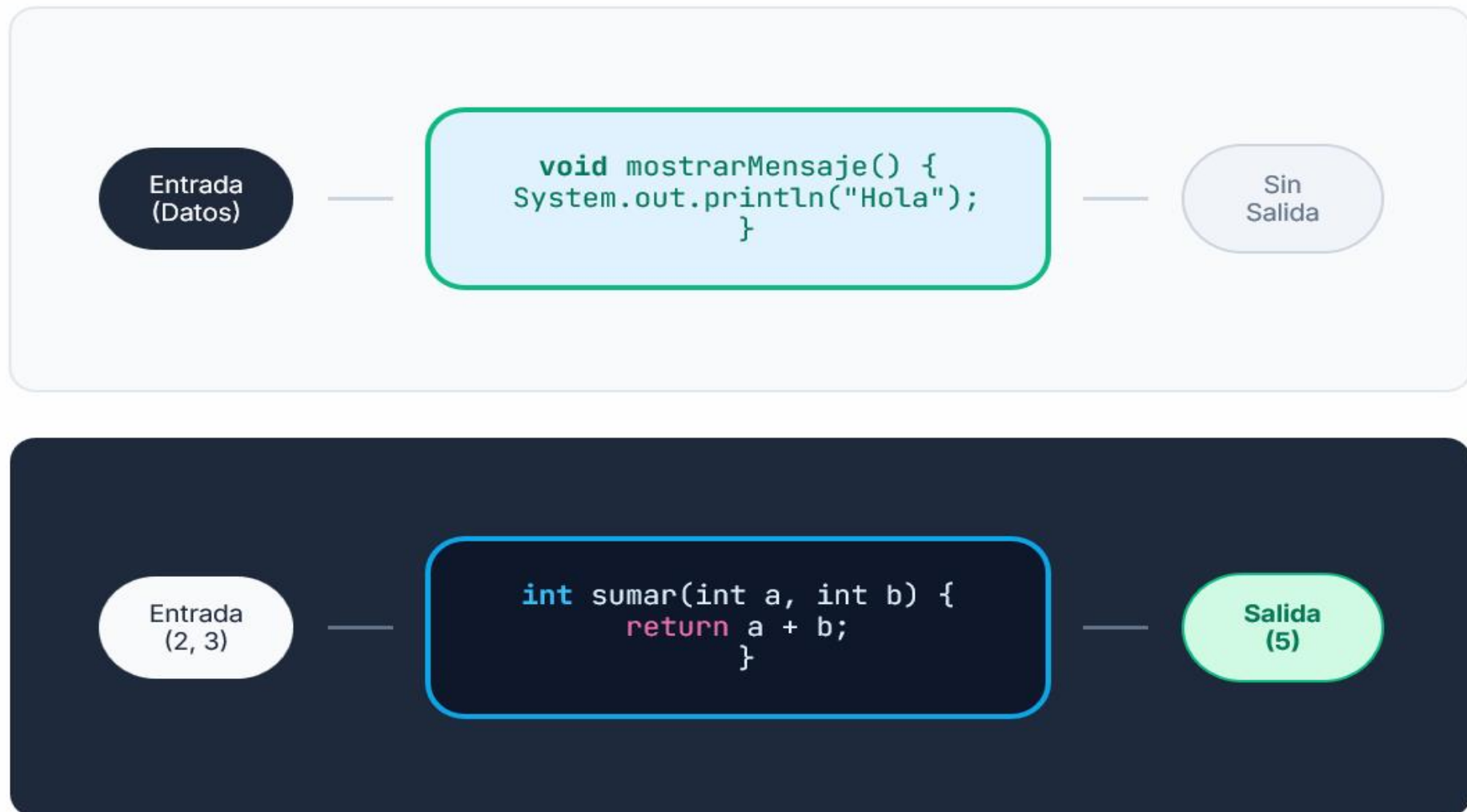
No devuelven ningún valor. Solo ejecutan instrucciones.



Métodos con Retorno

Deben especificar el tipo de dato que devolverán y usar `return`.

Comparación Visual



→ Entrada → Proceso → Salida

✓ Uso correcto de return



↔ Parámetros y Argumentos

Enviando y recibiendo información en los métodos

➡ Paso de Datos

Los métodos pueden necesitar datos externos para funcionar. Diferenciamos dos conceptos clave:



Parámetros

Variables definidas en la firma del método que **reciben** los datos.



Argumentos

Los **valores reales** que se envían al invocar o llamar al método.

🔌 Ejemplos Visuales

Invocación (Argumentos)

saludar("Ana")



Definición (Parámetros)

void saludar(String nombre)

sumar(3, 5)



int sumar(int a, int b)

registrarAnimal("Toby", 4)



void registrarAnimal(String n, int e)

Main.java

```
// El parámetro es 'nombre' (String)
public static void saludar(String nombre) {
    System.out.println("Hola, " + nombre);
}

public static void main(String[] args) {
    // El argumento es "Ana" (valor real)
    saludar("Ana");
}
```



← El Valor de Retorno (return)

Finalizando métodos y entregando resultados

¿Qué hace return?

La instrucción `return` finaliza la ejecución de un método y **entrega un valor** específico al código que lo llamó.

1. Coincidencia de Tipo

El valor retornado debe coincidir con el tipo de retorno declarado en la firma del método.

2. Fin de Ejecución

Cualquier código escrito después de un `return` (en el mismo bloque) nunca se ejecutará.

3. Uso en Asignaciones

El valor retornado suele guardarse en una variable o usarse directamente en otra expresión.

Flujo de Retorno: `calcularPromedio`

1. Llamada y Entrada (Argumentos)

```
double miNota = calcularPromedio(8.5, 9.5);
```



2. Ejecución del Método

```
public double calcularPromedio(double a, double b) {  
    double suma = a + b; // 18.0  
    return suma / 2; // 9.0  
}
```



3. Entrega del Resultado (Return)

```
miNota = 9.0;
```

El método termina y la variable recibe el valor.



▶ Invocación de Métodos

Cómo llamar un método desde el código principal o desde un objeto

📌 Llamar un Método

La invocación (o llamada) a un método ejecuta las instrucciones definidas en su bloque de código.



Métodos de Instancia

Se llaman a través de un objeto creado (instanciado) usando la sintaxis: `objeto.metodo()`



Métodos Estáticos (static)

Pertenecen a la clase en sí, no a un objeto. Se llaman usando: `Clase.metodo()`



Ejecución desde main

El método `main` es el punto de entrada, desde allí se inician las llamadas.

Paso de Argumentos:

Al invocar, se deben proporcionar valores (argumentos) que coincidan con los parámetros.

Formas de Invocación



1. Llamar a un Método de Instancia

```
// Se necesita instanciar (crear) un objeto primero
Persona p = new Persona();

// Llamada a través de la variable del objeto
p.saludar();
```



2. Llamar a un Método Estático

```
// No se necesita instanciar objeto
// Se llama directamente usando el nombre de la Clase
int resultado = Utilidades.sumar(2, 3);
System.out.println(resultado);
```





Static vs Instancia

Contexto de los métodos: ¿A quién pertenecen?

? ¿Cómo Elegir?

La palabra reservada **static** define si un método requiere un objeto para funcionar o si es global a la clase.



Métodos de Instancia:

Pertenecen al objeto. Pueden acceder a variables de instancia (atributos) y usan `this`.



Métodos Static:

Pertenecen a la clase. No requieren instanciar objetos. No pueden usar variables de instancia ni `this`.



Uso común de static:

Clases de utilidad (`Math`), métodos de fábrica y el método `main`.

Criterio rápido: ¿Depende de los datos del objeto?
Instancia. ¿Es un cálculo general? Static.

Comparación Visual



Instancia

Requiere un objeto creado con `new`.

```
// Definición
public void hacerSonido() {
    print(this.nombre + " ladra");
}

// Invocación
Perro miPerro = new Perro();
miPerro.hacerSonido();
```

Llamado a través de `variable.metodo()`.

Puede acceder a atributos del objeto.

Diferentes objetos pueden tener distintos estados.



Static

Pertenece a la clase misma.

```
// Definición
public static int max(int a, int b){
    return (a > b) ? a : b;
}

// Invocación
int mayor = Math.max(10, 20);
```

Llamado a través de `Clase.metodo()`.

No necesita instanciar la clase.

Ideal para funciones independientes del estado.



Sobrecarga de Métodos

Mismo nombre, diferente firma (parámetros)

Mismo Nombre, Distinta Firma

La **sobrecarga** (overloading) permite definir múltiples métodos con el mismo nombre dentro de una clase, siempre que su firma sea distinta.

-  **Mismo Nombre:** Agrupa comportamientos lógicamente idénticos (ej. sumar, imprimir).
-  **Diferentes Parámetros:** Java los distingue por la cantidad, el tipo o el orden de los argumentos.
-  **No por Retorno:** Cambiar solo el tipo de retorno no es suficiente y genera error de compilación.

Regla de Oro:

-  La firma de un método está compuesta por su nombre y su lista de parámetros.

Ejemplo Visual: Métodos sumar()

1. Dos enteros

```
int sumar(int a, int b) {  
    return a + b;  
}
```

2. Dos decimales (Cambio de Tipo)

```
double sumar(double a, double b) {  
    return a + b;  
}
```

3. Tres enteros (Cambio de Cantidad)

```
int sumar(int a, int b, int c) {  
    return a + b + c;  
}
```

Llamadas a los métodos:

```
sumar(5, 10); // Ejecuta método 1  
sumar(2.5, 3.1); // Ejecuta método 2  
sumar(1, 2, 3); // Ejecuta método 3
```

Caso Práctico: Organización

Creando un ZooApp con métodos modulares

ZooApp Modular

En un programa real, los métodos permiten dividir tareas complejas en bloques lógicos, facilitando su reutilización, lectura y prueba.



agregarAnimal()

Recibe parámetros para crear y registrar.



buscarAnimal()

Recibe un ID y retorna los datos encontrados.



calcularPromedio()

Procesa datos internos y retorna un resultado double.

Flujo de Invocación

`void agregarAnimal(...)`

Lógica de registro

`String buscarAnimal(...)`

Lógica de búsqueda

`double calcularPromedio()`

Cálculo interno

```
public static void main(String[] args) {  
  
    agregarAnimal("León", 5);  
  
    String a = buscarAnimal(1);  
  
    double p = calcularPromedio();  
  
}
```

→ Ejecuta y termina

↪ Retorna "León"

↪ Retorna 5.0



El método **main** actúa como el director principal, delegando tareas específicas a otros métodos bien definidos para mantener el código limpio.





Errores Comunes en Métodos

Problemas frecuentes al definir y llamar métodos en Java



¿Qué suele salir mal?



Olvidar el return

Si el método no es void, debe retornar un valor en todas las rutas posibles.



Parámetros mal ordenados

Al llamar el método, el orden de los argumentos debe coincidir exactamente.



Tipo de retorno incorrecto

Intentar devolver un String en un método declarado como int.



Confundir static e instancia

Llamar un método de instancia sin crear un objeto previo (error muy común).



Abusar de métodos largos

Métodos de cientos de líneas son difíciles de leer, probar y mantener.



Error Consecuencia Solución

Falta return en método no-void

```
int sumar(int a, int b) {  
    int res = a + b;  
}
```

→ Error de compilación

```
int sumar(int a, int b) { return a + b; }
```

Llamada incorrecta (Static)

```
public static void main(...) {  
    saludar(); // saludar no es static  
}
```

→ No compila

```
Clase obj = new Clase(); obj.saludar();
```

Orden de Argumentos

```
void setDatos(String nom, int edad)  
setDatos(25, "Ana");
```

→ Tipos incompatibles

```
setDatos("Ana", 25);
```



✓ Buenas Prácticas

Consejos para diseñar métodos limpios, reutilizables y fáciles de mantener

💡 Reglas de Oro

A Nombres Claros

Usa verbos descriptivos en camelCase. (ej. `calcularTotal`, no `calc` o `hacerTodo`).

🎯 Una Sola Responsabilidad

El método debe hacer solo una cosa y hacerla bien. Si hace varias, divídelo en métodos más pequeños.

✂️ Métodos Cortos

Mantén el cuerpo breve. Lo ideal es que la lógica principal no supere las 20-30 líneas de código.

• Parámetros Mínimos

Intenta usar 3 parámetros o menos. Si necesitas pasar más datos, agrúpalos en un solo objeto.

📋 Checklist Visual: Lo que SÍ debes hacer



Devolver Valores Útiles en vez de imprimir

```
return total; // En lugar de System.out.println(total);
```



Validar Parámetros de Entrada (Fail Fast)

```
if (precio < 0) return 0.0;
```



Usar JavaDoc para Documentación

```
/** @param id ID de usuario @return boolean */
```



Evitar Efectos Secundarios (Side effects)

No modifiques el estado global silenciosamente.



Resumen Visual: Métodos en Java

Hoja de repaso rápido de sintaxis, parámetros, retorno y características clave



Estructura Base

Sintaxis

Firma

Todo método requiere un tipo de retorno, un nombre descriptivo y paréntesis para los parámetros.

```
public int sumar(int a, int b) {  
    return a + b;  
}
```

- **public:** Modificador
- **int:** Retorno
- **sumar:** Nombre



Entrada y Salida

Parámetros

return

void

Los métodos reciben datos para procesar y opcionalmente devuelven un resultado al invocador.

```
// Con retorno  
int doble(int n) { return n * 2; }  
  
// Sin retorno (void)  
void saludar() { print("Hola"); }
```

- **return** termina el método
- **void** indica que no hay retorno



Características

static

Sobrecarga

Herramientas para organizar llamadas y crear métodos más flexibles.

```
// Sobrecarga (Overloading)  
void print(int x) { ... }  
void print(String s) { ... }  
  
// Static (de clase)  
Math.max(10, 20); // No requiere  
new
```

"La sobrecarga permite usar el mismo nombre si cambian los parámetros. Static asocia el método a la clase, no al objeto."





Conclusiones y Próximos Pasos

Cierre de Métodos en Java

Ideas Clave

Comprender el uso de **métodos** es fundamental para crear programas organizados y eficientes en Java.



Organización de Lógica

Un método agrupa instrucciones para realizar una tarea específica.



Datos y Resultados

Los parámetros reciben datos; `return` entrega resultados procesados.



Contexto y Claridad

`static` cambia el contexto (clase vs objeto). La claridad mejora el mantenimiento.

Resultado:

• Código más reutilizable, modular y legible

Estructura Fundamental



→ Próximos Pasos

1 Métodos con Objetos

2 Recursividad

3 Pruebas Unitarias



Métodos en Java



Buenas Prácticas



Felicitaciones
¡Has finalizado el módulo!

DuocUC[®]



CERCANÍA. LIDERAZGO. FUTURO.

7
AÑOS
ACREDITADO

 Comisión Nacional
de Acreditación
CNA-Chile

NIVEL DE EXCELENCIA
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la
calidad / Vinculación con el Medio / Investigación, creación y/o innovación